VIETNAM GENERAL CONFEDERATION OF LABOR

TON DUC THANG UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY



NGÔ LÂM TIẾN – 524H0195
NGUYỄN CÔNG THÀNH – 524H0191
NGUYỄN LÊ NHẬT TÂM – 524H0186

MIDTERM REPORT

INTRODUCTION TO MACHINE LEARNING

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



NGÔ LÂM TIẾN – 524H0195
NGUYỄN CÔNG THÀNH – 524H0191
NGUYỄN LÊ NHẬT TÂM – 524H0186

MIDTERM REPORT

INTRODUCTION TO MACHINE LEARNING

Advised by
Lê Anh Cường, Ph.D.

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENT

We thank Lê Anh Cường, Ph.D., for the course materials in Introduction to Machine Learning. These lectures provide the theoretical basis for the derivations, model implementations and evaluation procedures presented in this report.

The accompanying programs connect the mathematical definitions with reproducible experiments. We also acknowledge OpenML for providing the credit-g dataset and its metadata. The dataset makes it possible to study preprocessing, classification and overfitting on a common set of observations.

This report was prepared with AI assistance in drafting, programming and checking. The supporting source code, data provenance and experiment outputs are included so that the work can be inspected and reproduced.

Ho Chi Minh City,

October 14, 2026

Authors

Ngô Lâm Tiến

Nguyễn Công Thành

Nguyễn Lê Nhật Tâm

DECLARATION OF AUTHORSHIP

This report is prepared for the midterm assignment in Introduction to Machine Learning by Ngô Lâm Tiến (524H0195), Nguyễn Công Thành (524H0191) and Nguyễn Lê Nhật Tâm (524H0186).

Its theoretical content is restricted to the supplied course materials. The synthetic datasets in Question 1 are generated by the included code. The dataset in Question 2 is the official OpenML credit-g dataset, ID 31, version 1. Reported numerical results are outputs of the accompanying experiment programs; lecture examples are not presented as our experimental findings.

AI assistance was used in preparing the text, implementations and checks. The course slides, dataset source and use of libraries are acknowledged.

The authors are responsible for reviewing the derivations, code and final submitted version, and for complying with the course's rules on permitted assistance.

Ho Chi Minh City,
October 14, 2026
Authors
Ngô Lâm Tiến
Nguyễn Công Thành
Nguyễn Lê Nhật Tâm

TABLE OF CONTENT

CHAPTER 1. INTRODUCTION **1**

1.1 Objectives and scope 1

1.2 Experimental principles 1

CHAPTER 2. QUESTION 1: GRADIENT DESCENT **2**

2.1 Notation and data preparation 2

2.2 Linear regression 2

2.3 Logistic regression 4

2.4 Python implementation and verification 6

2.5 Linear regression experiment 7

2.6 Logistic regression experiment 9

CHAPTER 3. QUESTION 2: CLASSIFICATION **12**

3.1 Dataset and prediction task 12

3.2 Data audit, cleaning and visualization 12

3.3 Transformation, scaling and leakage control 14

3.4 Classification methods 15

3.5 Validation protocol and parameter selection 18

3.6 Test evaluation and comparison 21

CHAPTER 4. OVERFITTING AND REMEDIES **24**

4.1 Definition and experimental evidence 24

4.2 Decision-tree capacity experiment 25

4.3 Model-specific remedies and remaining risks 26

CHAPTER 5. REPRODUCIBILITY AND FINDINGS **29**

5.1 Files and execution 29

5.2 Findings and limits 29

REFERENCES **31**

LIST OF FIGURES

Figure 2.5.1: Scratch linear regression: training loss and test predictions. 8
Figure 2.6.1: Scratch logistic regression: training loss and test decision boundary. 10
Figure 3.2.1: Development-set class balance, numeric distributions and checking status. 13
Figure 3.6.1: Cross-validation and held-out macro F1 for all classifier entries. . . 22
Figure 3.6.2: MLP test confusion matrix and ROC curves of the three leading CV entries. 23
Figure 4.2.1: Tree depth: fold-training performance rises while validation eventually declines. 26

CHAPTER 1. INTRODUCTION

1.1 Objectives and scope

Question 1 requires linear and logistic regression from scratch: prediction, loss, gradient derivation, Gradient Descent (GD), training-loss plots and test evaluation. Chapter 2 addresses these requirements on two synthetic datasets.

For Question 2, Chapter 3 prepares and visualizes OpenML credit-g, then compares 14 learned classifier entries and a majority baseline. Chapter 4 analyzes overfitting; Chapter 5 gives reproducibility details.

The academic basis is limited to the course folder supplied by the students. In particular, linear regression follows Lecture 8; logistic regression follows Lecture 9; metrics and validation follow Lecture 3b and Lecture 4. The remaining classifiers are drawn from Lectures 3, 4, 5, 7, 9, 10 and 13. OpenML contributes data and provenance, rather than additional model theory.

1.2 Experimental principles

A low training loss does not establish generalization. The experiments separate learning from evaluation, record random seeds and fit preprocessing on training records only.

Question 1 uses NumPy for predictions, losses, gradients and updates; library utilities only split and evaluate data. Question 2 reuses scratch logistic regression and uses scikit-learn for the other classifiers.

For Question 2, a stratified 80/20 split reserves 200 records for testing. Model and parameter selection use identical five-fold stratified cross-validation within the remaining 800 records. Macro F1 is the selection metric because both classes should contribute to the comparison. All model choices are completed before any held-out test predictions are produced. The test results are reported for comparison, without retuning the models.

Course source: Lecture 1, pp. 4–11; Lecture 2, pp. 3–11; Lecture 3b, pp. 17–25; Lecture 4, pp. 34–37.

CHAPTER 2. QUESTION 1: GRADIENT DESCENT

2.1 Notation and data preparation

Let the training set be $\{(x_i, y_i)\}_{i=1}^{n}$, where $x_i \in \mathbb{R}^d$. Rows of $X \in \mathbb{R}^{n \times d}$ are the feature vectors x_i^{T}. Both models have feature weights $w \in \mathbb{R}^d$ and an intercept $b \in \mathbb{R}$. Let $\mathbf{1} \in \mathbb{R}^n$ denote a vector of ones and $\eta > 0$ the learning rate.

A train/test split is performed before fitting a transformation. For a numerical feature j, min–max scaling uses only the training minimum m_j and range r_j:

$$\tilde{x}_{ij} = \frac{x_{ij} - m_j}{r_j}, \qquad m_j = \min_{i \in train} x_{ij}, \quad r_j = \max_{i \in train} x_{ij} - m_j.$$

The code uses a denominator of 1 if a training feature is constant. Test values are transformed with the same training quantities and can lie outside $[0, 1]$. They are not clipped or used to recompute the scale. Scaling makes the GD step size easier to manage; it does not change the identity of the observations.

The derivations below use x_i for the features supplied to the model, which are the scaled features in the experiments. The raw-unit coefficients can be recovered as $w_j^{raw} = w_j / r_j$ and $b^{raw} = b - \sum_j w_j m_j / r_j$ when $r_j > 0$.

Course source: Lecture 3, pp. 18–25; Lecture 8, pp. 6–19.

2.2 Linear regression

2.2.1 Model, prediction and task

Linear regression predicts a continuous target from an affine combination of features:

$$\hat{y}_i = w^{\mathrm{T}} x_i + b, \qquad \hat{y} = Xw + b\mathbf{1}.$$

For a single feature, this becomes a straight line. For multiple features, it is a hyperplane. Unlike a binary classifier, the prediction is a real number without a probability constraint. Applications in the course include predicting weight or house price. The experiment below uses a synthetic continuous target with a known linear data-generating relation.

2.2.2 Loss function

The mean half-squared-error loss is

$$J_L(w, b) = \frac{1}{2n} \sum_{i=1}^{n} (w^{\mathrm{T}} x_i + b - y_i)^2.$$

The factor $1/2$ simplifies differentiation and does not change the minimizer. The division by n makes the loss an average. The evaluation metric MSE is $2J_L$ on the same observations, not J_L. Squaring makes large residuals contribute strongly, so extreme target errors can have a substantial effect on the fitted model.

2.2.3 Derivation of the gradients

Define $e_i = w^{\mathrm{T}} x_i + b - y_i$. For any feature weight w_j,

$$\frac{\partial e_i}{\partial w_j} = x_{ij}, \qquad \frac{\partial e_i}{\partial b} = 1.$$

Applying the chain rule to each squared residual gives

$$\frac{\partial J_L}{\partial w_j} = \frac{1}{2n} \sum_{i=1}^{n} 2e_i \frac{\partial e_i}{\partial w_j} = \frac{1}{n} \sum_{i=1}^{n} e_i x_{ij},$$

$$\frac{\partial J_L}{\partial b} = \frac{1}{2n} \sum_{i=1}^{n} 2e_i = \frac{1}{n} \sum_{i=1}^{n} e_i.$$

Writing $e = Xw + b\mathbf{1} - y$, the same gradients are

$$\nabla_w J_L = \frac{1}{n} X^{\mathrm{T}} e, \qquad \frac{\partial J_L}{\partial b} = \frac{1}{n} \mathbf{1}^{\mathrm{T}} e.$$

The vector gradient has d components and the intercept gradient is a scalar. These dimensions match the parameters that are updated.

2.2.4 Training algorithm

Initialize $w^{(0)} = 0$ and $b^{(0)} = 0$. At iteration t, predict using the current parameters and calculate the residuals

$$e^{(t)} = Xw^{(t)} + b^{(t)}\mathbf{1} - y.$$

Then perform the simultaneous updates

$$w^{(t+1)} = w^{(t)} - \eta \frac{1}{n} X^{\mathrm{T}} e^{(t)},$$
$$b^{(t+1)} = b^{(t)} - \eta \frac{1}{n} \mathbf{1}^{\mathrm{T}} e^{(t)}.$$

Both gradients are computed before either parameter is changed. Full-batch GD uses every training example at each iteration. A learning rate that is too large can cause divergence; one that is too small can make learning slow. The stopping rule uses the training gradient norm or a maximum iteration budget, never the test metrics.

Course source: Lecture 8, pp. 6–20 and 24.

2.3 Logistic regression

2.3.1 Model, prediction and task

Logistic regression is a binary classification model for labels $y_i \in \{0, 1\}$. It maps an affine score to a probability:

$$z_i = w^{\mathrm{T}} x_i + b, \qquad p_i = P(y_i = 1 \mid x_i) = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}.$$

Consequently $P(y_i = 0 \mid x_i) = 1 - p_i$. The probability is bounded between 0 and 1 for finite scores, whereas the score itself is unrestricted. Its log-odds are affine:

$$\log \frac{p_i}{1 - p_i} = w^{\mathrm{T}} x_i + b.$$

The experiment uses a fixed decision rule $\hat{y}_i = 1$ if $p_i \geq 0.5$, and $\hat{y}_i = 0$ otherwise. This corresponds to a boundary at $z_i = 0$. Probabilities, rather than thresholded labels, are used for ROC-AUC.

2.3.2 Appropriate loss and likelihood

For independent training observations, the Bernoulli likelihood is

$$\mathcal{L}(w, b) = \prod_{i=1}^{n} p_i^{y_i} (1 - p_i)^{1 - y_i}.$$

Maximizing this likelihood is equivalent to minimizing its negative mean log-likelihood, the binary cross-entropy loss:

$$J_G(w,b) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right].$$

Natural logarithms are used. A confident wrong prediction incurs a large loss. Squared error is therefore not substituted for the cross entropy taught for logistic regression.

2.3.3 Derivation of the gradients

First differentiate the sigmoid:

$$\begin{aligned}\sigma'(z) &= \frac{e^{-z}}{(1+e^{-z})^2} \\ &= \frac{1}{1+e^{-z}}\left(1-\frac{1}{1+e^{-z}}\right) = \sigma(z)(1-\sigma(z)).\end{aligned}$$

For one observation, let $\ell_i = -y_i \log p_i - (1-y_i)\log(1-p_i)$. Its derivative with respect to the probability is

$$\frac{\partial \ell_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i}.$$

Applying the chain rule through $p_i = \sigma(z_i)$ simplifies the derivative with respect to the score:

$$\begin{aligned}\frac{\partial \ell_i}{\partial z_i} &= \left(-\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i}\right)p_i(1-p_i) \\ &= -y_i(1-p_i) + (1-y_i)p_i = p_i - y_i.\end{aligned}$$

Finally, $\partial z_i/\partial w_j = x_{ij}$ and $\partial z_i/\partial b = 1$, hence

$$\frac{\partial J_G}{\partial w_j} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)x_{ij},$$

$$\frac{\partial J_G}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i).$$

In matrix form, with $p = \sigma(Xw + b\mathbf{1})$ applied elementwise,

$$\nabla_w J_G = \frac{1}{n}X^{\mathrm{T}}(p-y), \qquad \frac{\partial J_G}{\partial b} = \frac{1}{n}\mathbf{1}^{\mathrm{T}}(p-y).$$

The result resembles the linear-regression gradient, but the prediction function and loss differ.

2.3.4 Training and stable computation

Initialize both parameters at zero. At each iteration, compute z, p, J_G and both gradients using the current parameters, then update

$$w^{(t+1)} = w^{(t)} - \eta \frac{1}{n} X^{\mathrm{T}}(p^{(t)} - y),$$

$$b^{(t+1)} = b^{(t)} - \eta \frac{1}{n} \mathbf{1}^{\mathrm{T}}(p^{(t)} - y).$$

The code evaluates the sigmoid as $1/(1 + e^{-z})$ for $z \geq 0$ and $e^{z}/(1 + e^{z})$ for $z < 0$. The two expressions are equal, but this form avoids exponentiating a large positive number. Substituting the sigmoid into the cross entropy also gives

$$\ell_i = \log(1 + e^{z_i}) - y_i z_i.$$

NumPy's logaddexp operation evaluates the logarithmic term stably. These are numerical implementations of the same model and objective, not a change of loss.

The mean-loss gradient includes $1/n$. This follows Lecture 9, p. 42, and keeps the update consistent with the defined objective even though the summary update on p. 43 omits that factor.

Course source: Lecture 9, pp. 16–21 and 29–43.

2.4 Python implementation and verification

The accompanying file src/models.py contains the complete implementations. The linear loss function computes the residual vector, mean half-squared loss, matrix gradient and mean intercept gradient. The logistic loss function computes the stable logits-based cross entropy and the corresponding probability residual.

The two classes share a GD loop because only the prediction residual and loss change. The loop follows this procedure:

1. Validate finite two-dimensional features and a one-dimensional target; require 0/1 labels for logistic regression.
2. Initialize weights and intercept at zero. Store the loss before any update.
3. Compute the current training loss and both gradients. Reject non-finite values as a divergence error.

4. Stop if the full gradient norm is at most 10^{-8} or the iteration budget has been reached.
5. Otherwise update all parameters from those gradients and repeat. Record the resulting loss at the next iteration.

The history therefore includes iteration 0 and the final parameter state. Prediction checks that the feature dimension agrees with the fitted model. The scikit-learn adapter used in Question 2 delegates optimization to the same NumPy logistic implementation.

To verify the gradients independently, the tests compare each analytic derivative with a central finite difference of the loss:

$$g_j^{FD} = \frac{J(\theta + he_j) - J(\theta - he_j)}{2h}, \qquad h = 10^{-6},$$

where $\theta = (w_1, \ldots, w_d, b)$ and e_j is a coordinate unit vector. Both losses pass the comparison with relative tolerance 10^{-6} and absolute tolerance 10^{-8}. Additional tests fit an exact linear relation, classify separable examples, check probabilities sum to one, exercise logits of ± 1000, and reject invalid inputs. No finite-difference gradient is used for training.

2.5 Linear regression experiment

2.5.1 Dataset and training setup

The synthetic dataset has 600 examples with three independent features drawn uniformly from $[-3, 3]$, using NumPy seed 42. Its target is

$$y = 4 + 3x_1 - 2x_2 + 1.5x_3 + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, 0.5^2).$$

A shuffled 80/20 split with seed 42 produces 480 training and 120 test examples. The training features determine the min–max scale. GD starts from zero, uses $\eta = 0.3$, and has a maximum of 4,000 updates. It stops after 1175 updates because the training gradient norm falls below 10^{-8}.

2.5.2 Metrics and results

Let m denote the number of test examples. The regression metrics are

$$\text{MSE} = \frac{1}{m}\sum_i (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}},$$

$$\text{MAE} = \frac{1}{m}\sum_i |y_i - \hat{y}_i|, \qquad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y}_{test})^2}.$$

MSE has squared target units; RMSE and MAE have target units. The R^2 denominator uses the mean of the evaluated test targets, as in the lecture metric definition. It is an evaluation calculation, not a parameter fitted on the test set.

Table 2.5.1: Scratch linear-regression results.

Quantity	Value
Initial training loss	30.8891
Final training loss (MSE/2)	0.126302
Test MSE	0.2615
Test RMSE	0.5114
Test MAE	0.4045
Test R^2	0.9947

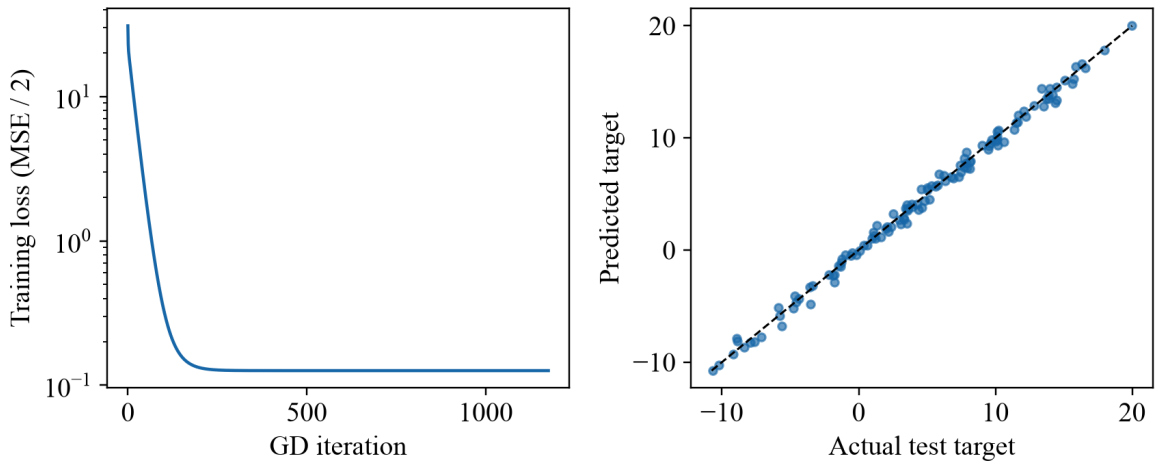


Figure 2.5.1: Scratch linear regression: training loss and test predictions.

The loss decreases from 30.8891 to 0.126302. On the original feature scale, the estimated coefficients are approximately $(2.9929, -2.0055, 1.4939)$ and the intercept is 3.9520, close to the generating values $(3, -2, 1.5)$ and 4. Test RMSE is 0.5114, which is close to the noise standard deviation of 0.5. The strong R^2 reflects the deliberately linear synthetic problem; it is not evidence of performance on arbitrary real datasets.

Course source: Lecture 3b, pp. 29–37; Lecture 8, pp. 6–24.

2.6 Logistic regression experiment

2.6.1 Dataset and training setup

A separate random generator with seed 43 creates 800 examples with two independent standard-normal features. First define

$$p^{true}(x) = \sigma(0.8 + 1.6x_1 - 1.2x_2),$$

then draw a Bernoulli label by comparing an independent uniform random number with this probability. Thus labels are probabilistic rather than perfectly separated by a line. This prevents a visually simple boundary from implying that every label must be predicted correctly.

A stratified 80/20 split with seed 42 gives 640 training and 160 test examples. The positive class has 393 training and 98 test examples. GD uses the training min–max scale, $\eta = 0.5$ and a maximum of 5,000 updates. The iteration budget is reached; the final gradient norm is about 1.55×10^{-4}, so the strict 10^{-8} stopping tolerance is not reached. Neither the budget nor threshold is chosen using the test results.

2.6.2 Metrics and results

For positive class 1, let TP, FP, FN and TN be the confusion counts. We use

$$\text{Accuracy} = \frac{TP + TN}{m}, \quad \text{Precision} = \frac{TP}{TP + FP},$$
$$\text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2TP}{2TP + FP + FN}.$$

Macro F1 averages the two class-specific F1 values. ROC-AUC evaluates the ordering induced by the predicted probabilities across thresholds. Undefined precision or F1 is assigned zero in the evaluation code.

Table 2.6.1: Scratch logistic-regression results (positive class 1).

Quantity	Value
Initial training cross entropy	0.693147
Final training cross entropy	0.452535
Test cross entropy	0.448082
Test accuracy	81.25%
Test precision	0.8269
Test recall	0.8776
Test positive-class F1	0.8515
Test macro F1	0.7986
Test ROC-AUC	0.8576

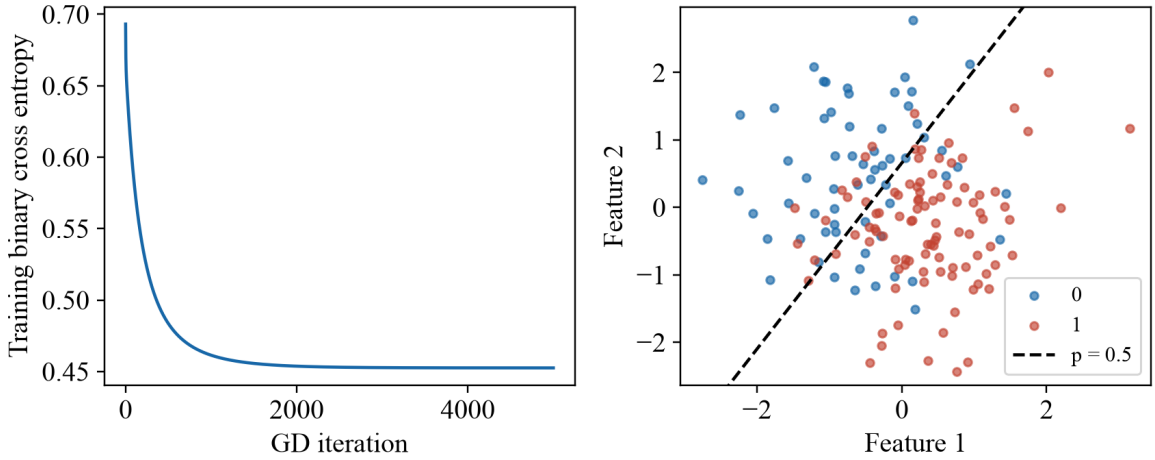


Figure 2.6.1: Scratch logistic regression: training loss and test decision boundary.

Rows of the test confusion matrix are actual classes $(0, 1)$ and columns are predicted classes $(0, 1)$:

$$C = \begin{pmatrix} 44 & 18 \\ 12 & 86 \end{pmatrix}.$$

There are 130 correct predictions among 160 examples. The estimated raw-unit coefficients are approximately $(1.4522, -1.0534)$ with intercept 0.6949. They follow the generating direction but differ because the labels are sampled and training is finite. Training cross entropy decreases from 0.693147 to 0.452535, while positive-class F1 is 0.8515 on the test set. The curve alone does not establish generalization; the separate test metrics provide the complementary evidence.

Course source: Lecture 3b, pp. 4–16; Lecture 4, pp. 34–36; Lecture 9, pp. 29–43.

CHAPTER 3. QUESTION 2: CLASSIFICATION

3.1 Dataset and prediction task

The dataset is *credit-g*, OpenML ID 31, version 1, also described as German Credit. Its official ARFF contains 1,000 records, 20 predictors and a class target with labels good and bad. The OpenML metadata credits Dr. Hans Hofmann and lists a public licence. The experiment preserves the downloaded ARFF and metadata alongside the code.

The target is mapped to $0 = good$ and $1 = bad$. There are 700 good and 300 bad records. The task is to predict this supplied classification label from the predictors; the experiment is an academic classification comparison on this historical dataset.

The seven numerical predictors are duration, credit amount, installment commitment, residence since, age, existing credits and number of dependents. The remaining 13 predictors are categorical: checking status, credit history, purpose, savings status, employment, personal status, other parties, property magnitude, other payment plans, housing, job, own telephone and foreign worker. No row identifier is used as a predictor.

Source: https://www.openml.org/d/31. The official data checksum is verified before loading. Its MD5 is 9a475053fed0c26ee95cd4525e50074c; the complete SHA-256 is preserved in results/q2_audit.json.

3.2 Data audit, cleaning and visualization

The loader decodes ARFF categorical values, recognizes the missing marker, checks the exact expected shape and target vocabulary, and audits missing cells, duplicate records and non-finite numeric values. The observed audit finds zero missing cells, zero duplicated rows and zero non-finite numeric values. Thus no record is deleted or imputed. Reporting this clean audit is more accurate than claiming that a cleaning operation removed records when none needed removal.

The train/test split is created before exploratory plots or learned transformations. All exploratory plots use the 800 development records, consisting of 560 good

and 240 bad examples. The 200 held-out records contain 140 good and 60 bad examples. During five-fold CV, each fold trains on 640 records and validates on 160; class proportions are preserved.

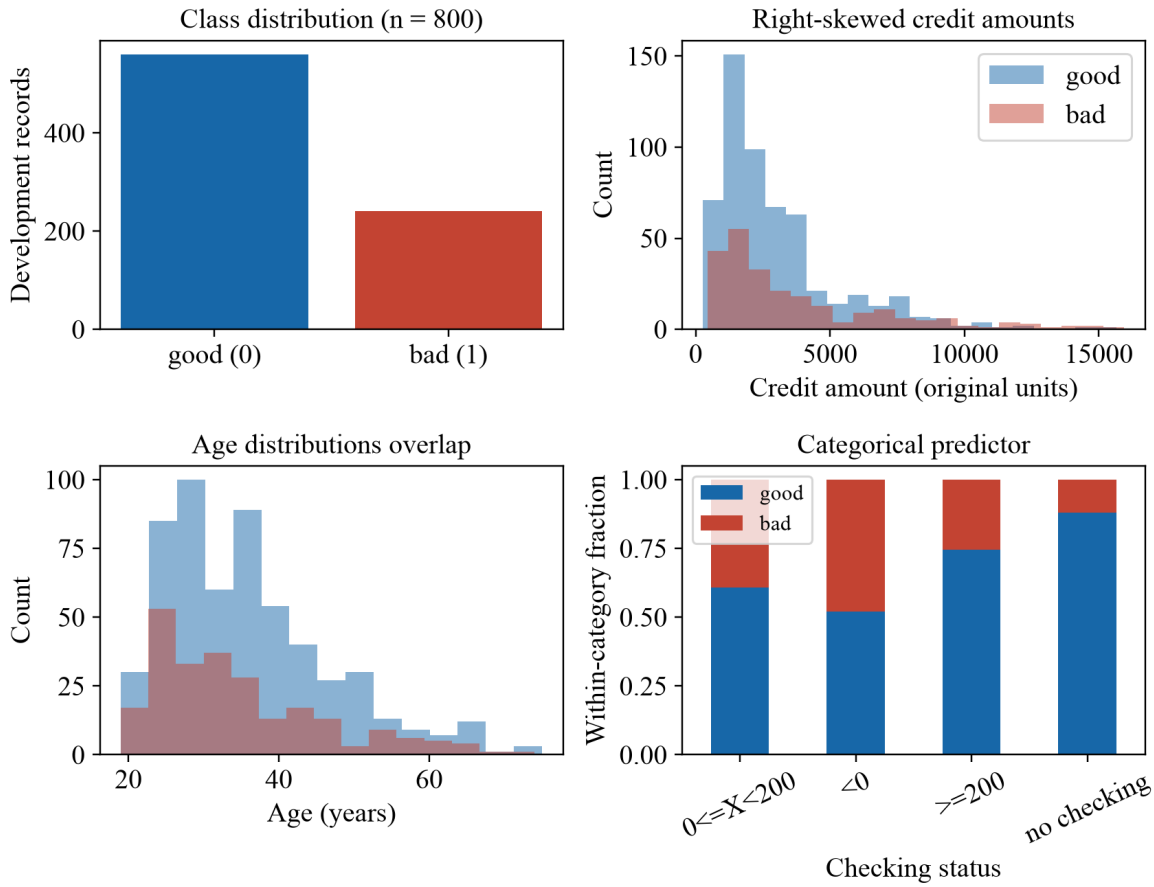


Figure 3.2.1: Development-set class balance, numeric distributions and checking status.

The class distribution is imbalanced. A classifier that always predicts good obtains 70% accuracy while detecting no bad examples. This makes accuracy alone an inadequate comparison criterion.

Credit amount is right-skewed: its development median is 2,317 and maximum is 15,945 in the original units. Age ranges from 19 to 75 years and the two class distributions overlap. Checking status shows different within-category class proportions. These plots motivate scaling, categorical encoding and multi-feature classification; they do not establish a causal relationship.

A development-only interquartile-range audit flags 58 credit-amount values and 25 ages outside the corresponding $[Q_1 - 1.5\,IQR, Q_3 + 1.5\,IQR]$ intervals. These are descriptive extreme-value flags, not verified data errors, so they are retained. The number-of-dependents feature is particularly instructive: its first and third quartiles are both 1, so this rule flags all 128 records with value 2. Deleting them merely because of this zero-IQR rule would remove legitimate categories. The complete numeric audit is saved for inspection.

Course source: Data Preparation for ML, pp. 2–6; Lecture 3, pp. 18–25; Lecture 3b, pp. 4–12.

3.3 Transformation, scaling and leakage control

Numerical features are min–max scaled using the minimum and maximum of each training fold. All categorical predictors are one-hot encoded, without imposing an artificial ordering on their values. All observed categories are retained; unknown categories at prediction time map to zero in the corresponding block. The transformation refitted on all 800 development records produces 61 numerical and indicator features in total.

The preprocessing transformer is placed inside the classification pipeline, so every CV fit learns its own ranges and category vocabulary from the fold's training subset. Validation and test data only pass through the fitted transformation. The target is separated first and is never encoded as an input feature. An independent test verifies that an extreme unseen age can transform to a value above 1 without changing the training maximum, and that an unseen category does not enter the fitted vocabulary.

The same numerical scaling and categorical encoding are used for all comparative models. This is essential for distance-based and margin-based methods; it is harmless for the threshold splits of tree models. Gaussian Naive Bayes models continuous densities on the resulting features, including indicators, so its Gaussian assumption is only approximate for binary columns. Bernoulli Naive Bayes instead keeps the indicators binary and converts each scaled numerical feature to $\mathbf{1}[\tilde{x}_j > 0.5]$.

This is a declared transformation that loses numeric detail, rather than an unreported change to the common dataset.

No oversampling, test-informed threshold selection, pretrained representation or additional feature source is used. In particular, the class imbalance is addressed through the evaluation criterion and class-specific reporting rather than through an unrequested resampling method.

Course source: Lecture 3, pp. 18–25; Lecture 4, pp. 37–42.

3.4 Classification methods

Fourteen learned classifier entries are compared. The three SVM kernels are variants of one family; they are not counted as independent algorithm families. The following descriptions identify the model mechanisms taught in the slides.

3.4.1 Linear classifiers and nearest neighbors

Scratch logistic regression uses the probability model and full-batch GD derived in Chapter 2. Its optimization is reused rather than replaced by a library logistic estimator. The 0.5 threshold remains fixed.

Perceptron forms $z = w^{\mathrm{T}}x + b$ and a hard classification boundary. With labels $s_i \in \{-1, +1\}$, the lecture update on a misclassified example is $w \leftarrow w + \eta s_i x_i$, with an analogous intercept update. The library performs the corresponding label handling. A finite number of passes is used because real labels need not be linearly separable.

kNN predicts by a majority vote among the k nearest training records under Euclidean distance. Numerical scaling prevents credit amount from dominating purely through its units. Uniform voting is used; candidate values of k explore the trade-off between local sensitivity and a smoother decision rule.

Course source: Lecture 3, pp. 4–25; Lecture 9, pp. 4–14 and 29–43.

3.4.2 Naive Bayes and decision trees

Naive Bayes chooses the class maximizing $P(c) \prod_j P(x_j \mid c)$, under conditional independence of features given the class. Gaussian NB estimates a mean and variance for each feature within each class. Bernoulli NB estimates probabilities for the binary

transformed features and uses additive smoothing. One-hot indicators from the same original predictor are dependent, so the independence assumption is a limitation of both comparisons.

Decision tree predicts by following feature tests from the root to a leaf. For a node with class proportions p_c, candidate split criteria include

$$H = -\sum_c p_c \log_2 p_c, \qquad G = 1 - \sum_c p_c^2.$$

The compared library tree uses binary threshold splits with entropy or Gini impurity. It is not presented as an exact implementation of categorical, multiway ID3. Maximum depth and minimum leaf size constrain its complexity.

Course source: Lecture 4, pp. 13–31 and 37–42; Lecture 5, pp. 18–34.

3.4.3 Support vector machines and neural network

SVM seeks a separating margin while allowing errors through the soft-margin parameter C. Kernel comparisons use the linear dot product, radial basis function and polynomial kernels:

$$K_L(x, z) = x^{\mathrm{T}} z,$$
$$K_R(x, z) = \exp(-\gamma \|x - z\|^2),$$
$$K_P(x, z) = (\gamma x^{\mathrm{T}} z + r)^q.$$

The polynomial experiment fixes $r = 0$ and compares degrees 2 and 3. The SVM decision score is used for ROC-AUC; it is not called a calibrated probability.

MLP is a feedforward neural network with one hidden layer and a nonlinear activation. Here it uses 16 or 32 tanh hidden units and a binary output. Backpropagation applies the chain rule to compute gradients; stochastic gradient descent updates the weights. This differs from the full-batch GD explicitly required and implemented in Question 1. The library uses a binary cross-entropy objective, so the quadratic-loss output derivative shown in Lecture 10 is not copied unchanged.

To keep optimization choices explicit, the MLP uses no weight penalty, no momentum and no early stopping. It trains for a fixed 500 epochs, with batch size at most

200. The result logs retain the maximum-iteration warnings. The report therefore does not claim that the MLP reaches a converged optimum.

Course source: Lecture 7, pp. 3–34; Lecture 10, pp. 3–32; Lecture 9, pp. 29–43.

3.4.4 Ensemble classifiers and scope of comparison

Bagging trains decision trees on bootstrap samples and aggregates their outputs. It reduces dependence on a single fitted tree. **Random forest** adds random feature selection at splits. The forest uses the square root of the encoded feature count as its feature-subset rule.

AdaBoost trains decision stumps sequentially, increases emphasis on misclassified observations and combines weighted weak classifiers. **Gradient boosting** builds an additive model by fitting successive trees to loss-derived residual information. The number of stages, learning rate and tree depth control the resulting complexity.

A **majority baseline** always predicts good and is included to show what can be obtained without learning feature–label relationships. It is an evaluation reference, not an additional lecture model.

CNN, RNN, LSTM and Transformer representations appear in the supplied deep-learning slides, but credit-g is a small tabular dataset without a supplied image, sequence or text representation. Imposing those structures would introduce an arbitrary representation and obscure the requested tabular comparison. Clustering is unsupervised and is not treated as a supervised classifier here. Multinomial NB is also omitted because these predictors are not the word-count data used to motivate it in the lectures.

Course source: Lecture 13, pp. 2–23; Lecture 11–12, pp. 16–134; Lecture 6, pp. 3–11.

3.5 Validation protocol and parameter selection

The comparison uses StratifiedKFold with five folds, shuffling and seed 42. All candidate pipelines receive the same fold indices. Parameters are selected by the mean validation macro F1,

$$F_M = \frac{F_{1,good} + F_{1,bad}}{2}.$$

For each configuration, training F1 and validation F1 are recorded on each fold. The sample standard deviation of the five validation scores is also reported. The training–validation gap is the difference between their means.

The finite search covers the following settings; the full per-candidate, per-fold results are included with the code:

- Scratch logistic: 1,500 or 4,000 GD updates at step 0.5. Perceptron: step 0.01, 0.1 or 1, with 300 passes.
- kNN: $k \in \{1, 5, 11, 21, 41\}$. Gaussian NB: variance smoothing 10^{-9}, 10^{-5} or 10^{-2}; this library safeguard adds a small variance term. Bernoulli NB: additive smoothing 0.5, 1 or 2.
- Tree: Gini or entropy, maximum depth 3, 6 or unlimited, and minimum leaf size 1 or 5.
- SVMs: $C \in \{0.1, 1, 10\}$; RBF compares scaled γ and 0.1; polynomial compares degree 2 and 3.
- MLP: hidden width 16 or 32, SGD learning rate 0.01 or 0.05, with the common 500-epoch budget.
- Bagging: depth 4 or unlimited and leaf size 1 or 5, with 50 trees. Random forest: depth 6, 10 or unlimited and leaf size 1 or 5, with 150 trees.
- AdaBoost: 50 or 150 stumps, step 0.1 or 1. Gradient boosting: 50 or 150 stages, step 0.05 or 0.1, depth 1 or 3.

The dataset split, candidate grids, scores and selected settings are saved. After selection, each model is refitted on all 800 development records and then evaluated on the

same 200 test records. In the tables, F_M denotes macro F1, SD is the sample standard deviation across folds, and all scores lie on the 0–1 scale.

Table 3.5.1: Model selection on development data (five folds).

Model	Train F_M	CV F_M	SD	Gap
Majority baseline	0.412	0.412	0.000	0.000
Scratch logistic	0.731	0.666	0.074	0.065
Perceptron	0.623	0.590	0.082	0.034
kNN	1.000	0.627	0.029	0.373
Gaussian NB	0.707	0.662	0.058	0.046
Bernoulli NB	0.716	0.694	0.064	0.022
Decision tree	0.777	0.656	0.067	0.121
Linear SVM	0.740	0.665	0.058	0.075
RBF SVM	0.990	0.669	0.066	0.321
Polynomial SVM	0.911	0.686	0.062	0.225
MLP	0.760	0.696	0.064	0.064
Bagging	0.901	0.680	0.053	0.221
Random forest	0.976	0.651	0.058	0.324
AdaBoost	0.736	0.664	0.056	0.072
Gradient boosting	0.882	0.665	0.027	0.217

The selected MLP has the largest validation mean, 0.6955, followed closely by Bernoulli NB at 0.6944 and polynomial SVM at 0.6862. Their fold-to-fold standard deviations are much larger than the difference between the first two means. This is a selection outcome on these folds, not evidence that MLP is statistically superior. Because the same CV results select parameters, they are not an independent, unbiased post-selection assessment. The held-out test set provides the separate final assessment.

Table 3.5.2: Selected settings, refitted on 800 records.

Model	Configuration
Majority baseline	Always predicts good; no tuned parameter.
Scratch logistic	$\eta = 0.5$, 1,500 full-batch updates, no penalty.
Perceptron	$\eta = 0.01$, 300 passes, shuffled samples.
kNN	$k = 1$, Euclidean distance, uniform voting.
Gaussian NB	Variance smoothing $= 0.01$; numeric and one-hot inputs.
Bernoulli NB	$\alpha = 2$; numeric threshold 0.5 after scaling.
Decision tree	Gini, maximum depth 6, minimum 5 records per leaf.
Linear SVM	Linear kernel, $C = 10$.
RBF SVM	$C = 10$; $\gamma = 1/(d \operatorname{Var}(X_{train}))$.
Polynomial SVM	$C = 1$, degree 3, offset 0; scaled γ.
MLP	One hidden layer of 16 tanh units, SGD step 0.05, 500 epochs.
Bagging	50 bootstrap trees, unlimited depth, minimum leaf size 5.
Random forest	150 trees, depth 10, minimum leaf size 1, $\sqrt{d}$ feature sampling.
AdaBoost	150 decision stumps, learning rate 1.
Gradient boosting	150 depth-3 trees, learning rate 0.05.

The scratch logistic classifier chooses 1,500 rather than 4,000 updates. A lower training loss after more iterations need not produce better validation classification at the fixed threshold. Similarly, kNN selects $k = 1$ despite a large training–validation gap. These choices are kept because selection follows the declared criterion; a visually smaller gap is not substituted for a better validation score.

3.6 Test evaluation and comparison

Accuracy, precision, recall and F1 are computed with bad as the positive class. Macro F1 gives equal weight to the two class F1 values. AUC uses probabilities for probabilistic models and decision scores for SVM/perceptron. The zero-division convention assigns zero where a precision or F1 denominator is zero.

Table 3.6.1: Held-out test metrics; positive class is bad.

Model	Acc.	P_1	R_1	$F_{1,1}$	F_M	AUC
Majority baseline	0.700	0.000	0.000	0.000	0.412	0.500
Scratch logistic	0.780	0.648	0.583	0.614	0.730	0.801
Perceptron	0.700	0.500	0.483	0.492	0.639	0.720
kNN	0.720	0.533	0.533	0.533	0.667	0.667
Gaussian NB	0.710	0.513	0.650	0.574	0.677	0.744
Bernoulli NB	0.735	0.556	0.583	0.569	0.689	0.778
Decision tree	0.645	0.429	0.550	0.482	0.606	0.692
Linear SVM	0.760	0.625	0.500	0.556	0.696	0.775
RBF SVM	0.745	0.576	0.567	0.571	0.695	0.799
Polynomial SVM	0.775	0.660	0.517	0.579	0.713	0.799
MLP	0.790	0.696	0.533	0.604	0.730	0.813
Bagging	0.730	0.568	0.417	0.481	0.649	0.752
Random forest	0.770	0.684	0.433	0.531	0.689	0.803
AdaBoost	0.785	0.681	0.533	0.598	0.726	0.790
Gradient boosting	0.775	0.660	0.517	0.579	0.713	0.783

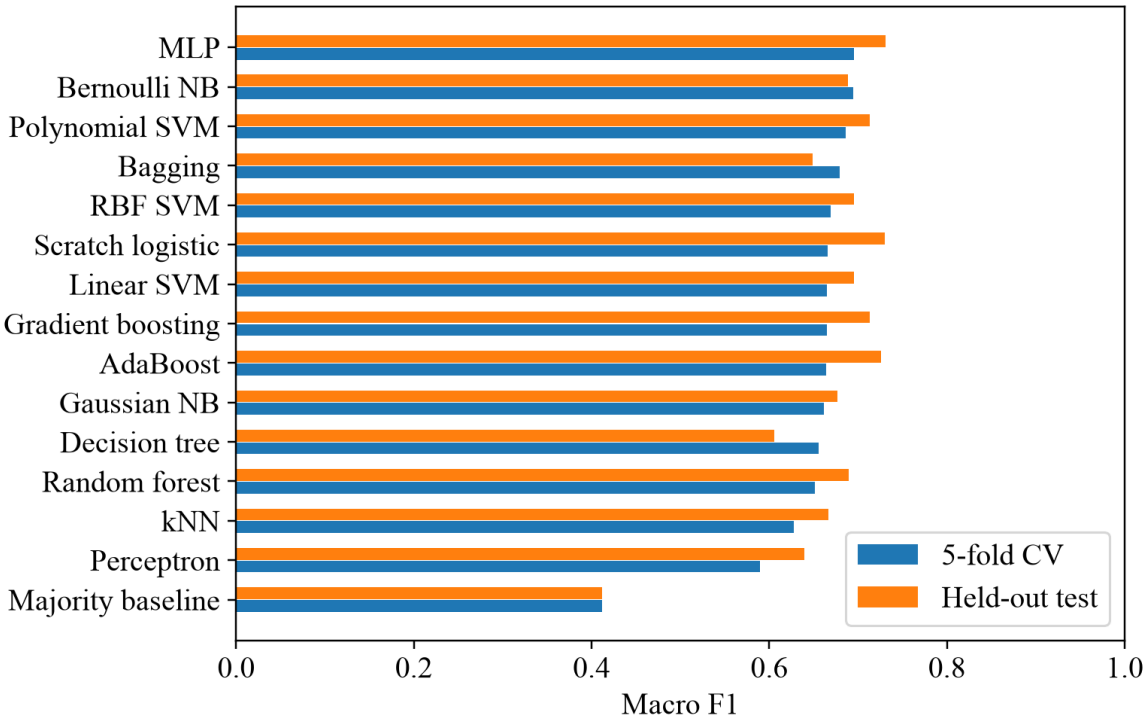


Figure 3.6.1: Cross-validation and held-out macro F1 for all classifier entries.

The majority baseline obtains 0.700 accuracy but 0 recall and F1 for bad, with macro F1 0.4118. Every learned classifier has higher test macro F1 than this reference, although the decision tree's accuracy is lower. This illustrates why the selected metric and class-specific results matter.

The CV-selected MLP attains 0.790 accuracy, 0.7305 macro F1 and 0.8133 AUC. Scratch logistic attains 0.780 accuracy and 0.7301 macro F1, nearly the same test macro F1. AdaBoost gives 0.785 accuracy and 0.7257 macro F1. The test observations are identical across models, and small differences should not be treated as universal rankings.

Gaussian NB has the highest bad-class recall among the learned entries, 0.650, but lower precision of 0.513. MLP has higher bad-class precision of 0.696 but recall of only 0.533. Thus the model with the best selection mean is not the best on every test metric. No threshold is subsequently adjusted to improve a preferred test score.

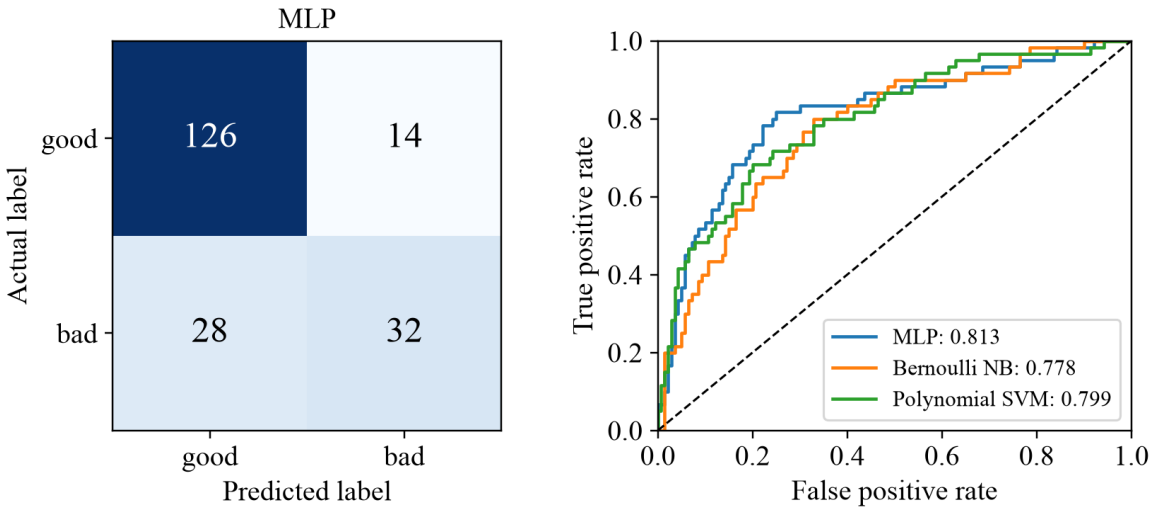


Figure 3.6.2: MLP test confusion matrix and ROC curves of the three leading CV entries.

For the selected MLP, the confusion matrix uses actual classes as rows and predicted classes as columns, both ordered good then bad:

$$C_{MLP} = \begin{pmatrix} 126 & 14 \\ 28 & 32 \end{pmatrix}.$$

This gives $TN = 126$, $FP = 14$, $FN = 28$ and $TP = 32$. In particular,

$$\text{Accuracy} = \frac{126 + 32}{200} = 0.79, \qquad \text{Recall}_{bad} = \frac{32}{60} = 0.5333.$$

Although 158 records are correctly classified, 28 of the 60 bad records are missed. The ROC curves show threshold-dependent ranking performance, but they do not remove this error pattern at the fixed operating rule. All 15 confusion matrices and prediction scores are preserved in the result files.

Course source: Lecture 3b, pp. 4–16 and 24–25; Lecture 4, pp. 34–36.

CHAPTER 4. OVERFITTING AND REMEDIES

4.1 Definition and experimental evidence

Overfitting occurs when a model fits the particular training sample too closely and performs substantially worse on unseen examples. Training performance can improve with greater complexity even when validation performance stops improving or declines. Conversely, a model with poor training and validation performance can be underfitting. The training–validation gap is useful evidence, but not a standalone proof that every error is due to overfitting.

Table 4.1.1 compares base and CV-selected configurations on the same folds. "Base" means the explicitly defined starting estimator in the experiment code; it does not mean that every parameter is the library's default. All four columns are mean macro F1, not accuracy.

Table 4.1.1: Base and selected configurations on identical folds.

Model	Base train	Base CV	Selected train	Selected CV
Majority baseline	0.412	0.412	0.412	0.412
Scratch logistic	0.731	0.665	0.731	0.666
Perceptron	0.623	0.590	0.623	0.590
kNN	0.750	0.610	1.000	0.627
Gaussian NB	0.656	0.616	0.707	0.662
Bernoulli NB	0.717	0.685	0.716	0.694
Decision tree	1.000	0.614	0.777	0.656
Linear SVM	0.733	0.656	0.740	0.665
RBF SVM	0.805	0.645	0.990	0.669
Polynomial SVM	0.815	0.656	0.911	0.686
MLP	0.451	0.437	0.760	0.696
Bagging	1.000	0.672	0.901	0.680
Random forest	1.000	0.641	0.976	0.651

Model	Base train	Base CV	Selected train	Selected CV
AdaBoost	0.713	0.646	0.736	0.664
Gradient boosting	0.911	0.641	0.882	0.665

The unrestricted base tree reaches training macro F1 of 1.000 but validation macro F1 of 0.614. The selected depth-6, minimum-leaf-5 tree has training macro F1 of 0.777 and validation macro F1 of 0.656. Its gap decreases from about 0.386 to 0.121. This is a direct experiment showing that limiting tree complexity can reduce overfitting and improve validation performance.

Bagging's gap falls from about 0.328 to 0.221 when a minimum leaf size of 5 is selected. Gradient boosting's selected setting lowers the gap from about 0.270 to 0.217. Random forest still has a large selected gap of about 0.324: ensembling does not guarantee that a model ceases to overfit.

4.2 Decision-tree capacity experiment

To isolate the effect of depth, an additional diagnostic varies the maximum depth of a Gini tree while keeping minimum leaf size 1, preprocessing, random seed and folds unchanged. It uses development data only and does not select a new model after viewing the test set.

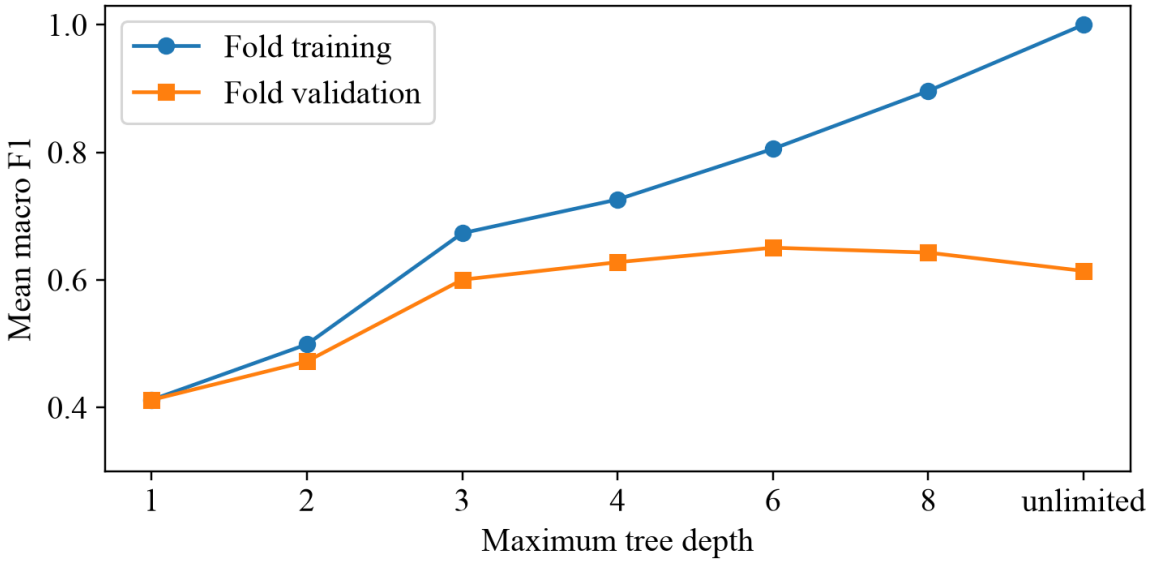


Figure 4.2.1: Tree depth: fold-training performance rises while validation eventually declines.

At depth 1, both means are approximately 0.412, consistent with a weak, majority-like model. At depth 6, training and validation macro F1 are about 0.805 and 0.650. At unlimited depth, training rises to 1.000 while validation falls to 0.614. The curve provides the expected pattern of increased capacity fitting training-specific structure.

This diagnostic differs from the full parameter search, which also varies leaf size and split criterion. It is therefore not expected to reproduce the selected tree's score exactly. A smaller depth or larger leaf size is useful only if it gives a suitable validation trade-off; extreme simplification can underfit.

Course source: Lecture 2, pp. 3–11; Lecture 5, pp. 27–34; supplementary decision-tree slides, pp. 29–36.

4.3 Model-specific remedies and remaining risks

Linear and logistic regression. Use a suitable feature set, inspect erroneous extreme values and validate the iteration budget rather than assuming that a lower training loss implies a better classifier. In Question 2, the unpenalized scratch logistic model

has a moderate gap of 0.065, with 1,500 updates selected. The experiment does not introduce a penalty absent from the required Question 1 objective. Synthetic Question 1 fits should not be generalized to real-world data.

Perceptron. Inspect whether a linear boundary is adequate and keep a finite training budget on nonseparable data. Its small gap of 0.034 accompanies a relatively low validation mean of 0.590; this is not evidence of a well-fitting model. Improving representation or reconsidering the linear model is more appropriate than treating all low scores as overfitting.

kNN. Increase k to reduce sensitivity to individual records and evaluate the resulting validation score. However, this dataset's search selects $k = 1$, whose training score is 1.000 and gap is 0.373. The larger-k candidates are recorded, and the reported selection is not overwritten solely to obtain a smaller gap. A smoother neighbor rule is an available remedy, not a demonstrated improvement in the chosen criterion here.

Gaussian and Bernoulli NB. Their gaps are about 0.046 and 0.022. Check the distribution assumptions and encoding rather than assuming that complex fitting is the main problem. Bernoulli smoothing avoids zero probabilities; numerical binarization loses information. One-hot blocks violate conditional independence, while Gaussian indicators violate the continuous Gaussian shape. These limitations can explain errors even without a large gap.

Decision tree. Limit depth, require more records per leaf, or prune using validation information, as discussed in the slides. The depth and leaf-size search gives the clearest measured reduction in overfitting. Its held-out performance remains weaker, so the mitigation is not described as solving every error.

SVM. A smaller C permits more training violations and can favor a less sample-specific boundary. Kernel choice, polynomial degree and RBF γ control flexibility and should be chosen using validation. The selected linear SVM gap is 0.075, compared with 0.321 for RBF and 0.225 for polynomial SVM. The selected nonlinear variants retain a risk of fitting details of the training sample even though their validation means are higher.

MLP. Use a smaller hidden layer and select training duration using validation rather than test performance. The search chooses 16 hidden units instead of 32 and a step size of 0.05. Its gap is approximately 0.064. The base network has both low training and validation scores; increasing the learning rate improves fitting substantially. The fixed 500-epoch run produces maximum-iteration warnings, so insufficient optimization and overfitting must be distinguished. A future validation-based duration comparison would require a fresh evaluation protocol, not retrospective test tuning.

Bagging and random forest. Bootstrap aggregation and random feature subsets reduce reliance on a single tree, while maximum depth and minimum leaf size further constrain each base learner. Their selected gaps remain 0.221 and 0.324. Diversity and averaging help, but strong trees can still memorize the finite development sample.

AdaBoost and gradient boosting. Validate the number of stages, learning rate and weak-learner complexity. Deep trees or too many corrective stages can fit sample-specific errors. AdaBoost here uses decision stumps and has a gap of 0.072; gradient boosting uses depth-3 trees and has a gap of 0.217. Reducing step size or tree complexity can be considered within a training-only search; it is not guaranteed to increase validation performance.

Across all models, the common safeguards are an untouched test set, preprocessing fitted within folds, a declared selection criterion and transparent reporting of training and validation performance. These safeguards improve the credibility of evaluation, but do not themselves make a model more accurate. The observed remedies are distinguished from possible next steps so that no unrun improvement is presented as an experimental result.

Course source: Lecture 3, pp. 4–28; Lecture 4, pp. 13–31; Lecture 5, pp. 27–34; Lecture 7, pp. 3–34; Lecture 10, pp. 18–32; Lecture 13, pp. 10–23.

CHAPTER 5. REPRODUCIBILITY AND FINDINGS

5.1 Files and execution

The project contains the scratch model code, experiment runner, tests, original OpenML ARFF and metadata, generated metrics, per-fold scores, split indices, predictions and figures. The LaTeX source uses native equations and generates the numerical tables from the stored result files. The README provides the complete run and build commands.

The experiment environment records Python 3.14.7, NumPy 2.5.3, pandas 3.0.6, SciPy 1.18.1, scikit-learn 1.9.1 and Matplotlib 3.11.2. These exact versions are recorded in the requirements file. Random seeds are fixed; single-thread settings are used for the supplied run. Small floating-point differences across platforms are possible, so the dataset checksum, split indices and saved predictions are the concrete reference artifacts.

The unit checks independently verify both loss gradients, an exact regression relation, stable extreme-logit behavior, invalid-input handling and train-only preprocessing. The artifact checks recompute every exported classifier test metric from its saved predictions and validate that the 800 development and 200 test row IDs are disjoint. A rerun after the final code corrections passes all six tests.

5.2 Findings and limits

For Question 1, linear regression learns the known synthetic relation with test $R^2 = 0.9947$ and RMSE 0.5114. Logistic regression reaches test accuracy 81.25%, positive-class F1 0.8515 and AUC 0.8576. Their analytic gradients agree with finite differences, and both training curves decrease under the specified learning rates.

For Question 2, MLP is selected using the highest mean development macro F1, then evaluated without further tuning. Its test accuracy is 0.790 and macro F1 is 0.7305, but it detects only 32 of 60 bad examples. Scratch logistic provides nearly the same test macro F1 with a simpler affine model. The small differences and fold variability do not support a claim of universal or statistically established superiority.

The decision-tree experiment shows overfitting directly: unlimited depth obtains a perfect training score with a lower validation score than a constrained tree. Other methods show different combinations of capacity, optimization difficulty and distribution assumptions. The report therefore treats overfitting as an observed generalization issue rather than an explanation for every poor result.

The main limits are one historical dataset, one fixed held-out split, modest parameter grids and finite GD/MLP training budgets. The CV means are used for selection and can be optimistic after searching. No independent replication dataset, nested validation or significance claim is added. The report fulfills the requested model derivations, implementations, preprocessing, comparisons and overfitting analysis within the supplied course scope.

REFERENCES

Lê Anh Cường. (2024). *Introduction to Machine Learning* (Lecture 1). Ton Duc Thang University. Supplied course slides.

Lê Anh Cường. (n.d.). *Principle of Machine Learning* (Lecture 2). Ton Duc Thang University. Supplied course slides, pp. 3–11.

Lê Anh Cường. (n.d.). *kNN Algorithm* (Lecture 3). Ton Duc Thang University. Supplied course slides, pp. 4–28.

Lê Anh Cường. (n.d.). *Evaluation of Machine Learning Models* (Lecture 3b). Ton Duc Thang University. Supplied course slides, pp. 4–37.

Lê Anh Cường. (2022). *Naive Bayes Classification* (Lecture 4). Ton Duc Thang University. Supplied course slides, pp. 13–42.

Lê Anh Cường. (n.d.). *Decision Tree* (Lecture 5 and supplementary "Decision tree – new" slides). Ton Duc Thang University. Supplied course slides.

Lê Anh Cường. (2022). *Unsupervised Learning and Clustering* (Lecture 6). Ton Duc Thang University. Supplied course slides, pp. 3–11.

Lê Anh Cường. (n.d.). *Support Vector Machines* (Lecture 7; file "SVM 2024"). Ton Duc Thang University. Supplied course slides.

Lê Anh Cường. (n.d.). *Linear Regression and Gradient Descent Algorithm* (Lecture 8). Ton Duc Thang University. Supplied course slides, pp. 6–25.

Lê Anh Cường. (n.d.). *Perceptron and Logistic Regression* (Lecture 9). Ton Duc Thang University. Supplied course slides, pp. 4–43.

Lê Anh Cường. (n.d.). *Neural Networks and Backpropagation Algorithm* (Lecture 10). Ton Duc Thang University. Supplied course slides, pp. 3–32.

Lê Anh Cường. (n.d.). *Deep Learning Introduction* (Lectures 11–12). Ton Duc Thang University. Supplied course slides, pp. 16–134.

Lê Anh Cường. (n.d.). *Ensemble Learning* (Lecture 13). Ton Duc Thang University. Supplied course slides, pp. 2–23.

Lê Anh Cường. (n.d.). *Data Preparation for ML*. Supplied course slides, pp. 2–17.

Hofmann, H. (Creator). (n.d.). *credit-g* [Dataset, version 1, OpenML ID 31]. OpenML. Retrieved October 14, 2026, from https://www.openml.org/d/31. Dataset description and checksum: https://www.openml.org/api/v1/json/data/31.